📘 Informe Técnico-Didáctico

CRUD en ASP.NET Core MVC con C#

Modo oscuro · Alivio visual

📑 Índice

- 1. Introducción al Patrón MVC
- 2. Estructura del Proyecto
- 3. Modelo: La Clase Datos
- 4. Vistas: La Capa de Presentación
- 5. Controlador: La Lógica de Negocio
- 6. Flujo Completo del CRUD
- 7. Análisis de Funcionamiento
- 8. Buenas Prácticas y Mejoras
- 9. Resumen Visual
- 10. Conclusión

1. Introducción al Patrón MVC

🔰 FUNDAMENTAL **MVC (Model-View-Controller)** es un patrón de arquitectura de software que separa una aplicación en tres componentes principales:

```
┌──────────────────────────────────────────────────────────┐
│                          MVC                             │
├──────────────────────────────────────────────────────────┤
│  MODELO      │   VISTA      │   CONTROLADOR              │
│  (Datos)     │ (Interfaz)   │   (Lógica)                 │
├──────────────┼──────────────┼────────────────────────────┤
│  • Datos     │ • HTML       │ • Recibe peticiones        │
│  • Reglas    │ • CSS        │ • Procesa datos            │
│  • Lógica    │ • JavaScript│ • Coordina M y V           │
│    negocio   │              │                            │
└──────────────┴──────────────┴────────────────────────────┘
```

✅ **Beneficios del Patrón MVC**

- **Separación de responsabilidades** – Cada componente tiene una función específica.
- **Mantenibilidad** – Facilita modificaciones sin afectar otras partes.
- **Reutilización** – Los componentes pueden reutilizarse.
- **Escalabilidad** – Permite crecimiento ordenado de la aplicación.

2. Estructura del Proyecto

```
EjercicioCompleto/
│
├── Models/
│   └── Datos.cs                # MODELO: Define la estructura de datos
│
├── Views/
│   ├── Home/
│   │   ├── Index.cshtml        # VISTA: Login/Inicio de sesión
│   │   ├── Agregar.cshtml      # VISTA: Listado y creación de usuarios
│   │   ├── Editar.cshtml       # VISTA: Formulario de edición
│   │   └── Eliminar.cshtml     # VISTA: Confirmación de eliminación
│   └── Shared/
│       └── _Layout.cshtml      # VISTA: Plantilla común
│
└── Controllers/
    └── HomeController.cs       # CONTROLADOR: Lógica de la aplicación
```

3. Modelo: La Clase Datos

```
namespace EjercicioCompleto.Models
{
    public class Datos
    {
        public string? usuario_ { get; set; }      // Nombre de usuario
        public string? contraseña_ { get; set; }   // Contraseña
        public string? habilitado_ { get; set; }   // Estado (on/null)
    }
}
```

Propiedades explicadas:

Propiedad	Tipo	¿Qué almacena?	¿Por qué string?
`usuario_`	`string?`	Nombre del usuario	Identificador único
`contraseña_`	`string?`	Contraseña del usuario	Almacena en texto plano (mejorable)
`habilitado_`	`string?`	Estado del usuario	"on" = habilitado, null = deshabilitado

💡 **Nota:** El checkbox HTML envía `"on"` cuando está marcado y `null` cuando no.

4. Vistas: La Capa de Presentación

📄 _Layout.cshtml (Plantilla común)

```
<html>
<head><title>@ViewData["Title"]</title></head>
<body>
    <header><nav>...</nav></header>
    <main>@RenderBody()</main>
    <footer>...</footer>
</body>
</html>
```

🔐 Index.cshtml (Login)

```
<form asp-controller="Home" asp-action="validar" method="post">
    <input type="text" name="txtUsuario" />
    <input type="password" name="txtContraseña" />
    <input type="submit" value="Ingresar" />
</form>
<h1>@ViewBag.respuesta</h1>
```

📋 Agregar.cshtml (Listado + Creación)

Combina un formulario para añadir usuarios y una tabla que muestra todos los registros con botones Editar/Eliminar.

```
<table>
    @foreach (var usuarios in ViewBag.respuesta)
    {
        <tr>
            <td>@usuarios.usuario_</td>
            <td>@usuarios.contraseña_</td>
            <td>@(usuarios.habilitado_ == "on" ? "HABILITADO" : "DESHABILITADO")</td>
            <td>
                <form asp-action="Editar" method="post">
                    <input type="hidden" name="txtEditarUsuario" value="@usuarios.usuario_" />
                    <input type="submit" value="Editar" />
                </form>
            </td>
            <td>
                <form asp-action="Eliminar" method="post">
                    <input type="hidden" name="txtUsuarioEliminar" value="@usuarios.usuario_" />
                    <input type="submit" value="Eliminar" />
                </form>
            </td>
        </tr>
    }
</table>
```

✏️ Editar.cshtml

Campo de usuario readonly (identificador), contraseña editable y checkbox para estado.

```
<input type="text" name="txtUsuarioNuevo" value="@ViewBag.UsuarioEditado" readonly />
<input type="password" name="txtContraseñaNuevo" value="@ViewBag.ContraseñaEditado" />
<input type="checkbox" name="chkHabilitadoNuevo" @ViewBag.HabilitadoEditado />
```

🗑️ Eliminar.cshtml

Confirmación antes de eliminar con dos formularios: Cancelar y Confirmar.

5. Controlador: La Lógica de Negocio

```
public class HomeController : Controller
{
    static List<Datos> listado = new List<Datos>();  // Almacenamiento en memoria

    // Index, validar, Agregar, Nuevo, Editar, EditarDatos, Eliminar, confirmaEliminar, confirmaVolver
}
```

> ⚠️ **Importante:** La lista `static` persiste en memoria mientras la app esté ejecutándose. Al reiniciar el servidor se pierden los datos.

🔍 validar() – Autenticación

```
var traeDatos = listado.Find(l => l.usuario_ == txtUsuario);
if (traeDatos == null) → "Usuario no existe"
else if (traeDatos.contraseña_ != txtContraseña) → "Contraseña incorrecta"
else if (traeDatos.habilitado_ != "on") → "Usuario no habilitado"
else → "Bienvenido"
```

➕ Nuevo() – Crear usuario

```
listado.Add(new Datos() { usuario_ = txtUsuarioNuevo, contraseña_ = txtContraseñaNuevo, habilitado_ = chkHabilitado });
```

✏️ EditarDatos() – Actualizar

```
var editaDatos = listado.Find(l => l.usuario_ == txtUsuarioNuevo);
editaDatos.contraseña_ = txtContraseñaNuevo;
editaDatos.habilitado_ = chkHabilitadoNuevo;
```

❌ confirmaEliminar() – Eliminar

```
var eliminaDatos = listado.Find(l => l.usuario_ == txtUsuarioDatoEliminar);
listado.Remove(eliminaDatos);
```

6. Flujo Completo del CRUD

```
CREATE (Crear)
Usuario → Agregar.cshtml → Nuevo() → listado.Add() → Retorna Agregar con lista actualizada

READ (Leer)
Agregar.cshtml → ViewBag.respuesta (lista) → @foreach muestra todos

UPDATE (Editar)
Click Editar → Editar() (ViewBag con datos) → Editar.cshtml → EditarDatos() → actualiza

DELETE (Eliminar)
Click Eliminar → Eliminar() → Eliminar.cshtml → Confirmar → confirmaEliminar() → Remove
```

7. Análisis de Funcionamiento

📦 ViewBag

Objeto dinámico que permite pasar datos del controlador a la vista. Ejemplo: `ViewBag.respuesta = listado;` y en vista `@ViewBag.respuesta`.

🎯 Tag Helpers

`asp-controller` y `asp-action` generan rutas automáticas: `/Home/validar`.

🕵️ Inputs Hidden

Envían datos al servidor sin mostrarlos (útil para IDs).

⚡ Condicionales en Vista

Operador ternario: `@(usuarios.habilitado_ == "on" ? "HABILITADO" : "DESHABILITADO")`

8. Buenas Prácticas y Mejoras

🔐 **Seguridad:** Usar hash de contraseñas (BCrypt).
🗄️ **Persistencia:** Reemplazar lista estática por Entity Framework Core con SQL Server/SQLite.
🆔 **Identificador:** Usar Id numérico autoincremental en lugar del nombre de usuario.
✅ **Validaciones:** DataAnnotations (`[Required]` , `[StringLength]`).
⚠️ **Excepciones:** Try-catch en operaciones críticas.

Ejemplo de modelo mejorado:

```
public class Usuario
{
    public int Id { get; set; }
    public string NombreUsuario { get; set; }
    public string ContraseñaHash { get; set; }
    public bool EstaHabilitado { get; set; }
    public DateTime FechaCreacion { get; set; }
}
```

9. Resumen Visual del Flujo

```
_Layout.cshtml
 ├── Inicio sesión → Index.cshtml → validar() → mensaje
 └── Agregar → Agregar.cshtml
       ├── Formulario Nuevo → Nuevo() → retorna Agregar
       ├── Tabla de usuarios
       │     ├── Editar → Editar.cshtml → EditarDatos() → actualiza
       │     └── Eliminar → Eliminar.cshtml → confirmaEliminar() → elimina
       └── (Vuelve siempre a Agregar.cshtml para mostrar lista actualizada)
```

10. Conclusión

Este proyecto es un ejemplo completo y funcional de un CRUD usando el patrón MVC en ASP.NET Core con C#. Se han implementado las cuatro operaciones fundamentales (Create, Read, Update, Delete) utilizando una lista en memoria como almacenamiento temporal, y se ha logrado una separación clara entre la lógica de negocio (controlador), la presentación (vistas) y los datos (modelo).
✅ **Fortalezas:** Estructura MVC clara, CRUD completo, uso de ViewBag, formularios con Tag Helpers, navegación coherente.
🔧 **Áreas de oportunidad:** Migrar a base de datos real, implementar hash de contraseñas, agregar validaciones robustas, manejo de errores y repositorios para escalabilidad.

📘 Informe generado con estilo Dark Mode para aliviar la vista.
Tecnología: ASP.NET Core MVC · C# · Razor Views
Propósito: Educación y comprensión del patrón MVC en aplicaciones CRUD.